

1. Explore pre-trained word vectors. Explore word relationships using vector arithmetic. Perform arithmetic operations and analyze results.

```
#!/pip install gensim
import gensim.downloader as api

model = api.load("word2vec-google-news-300")
print("\nSimilar to 'king':")
for w, s in model.most_similar("king", topn=5):
    print(w, round(s,4))
res = model.most_similar(positive=['king','woman'], negative=['man'], topn=1)
print("\nking - man + woman =", res[0][0])

sim = model.similarity("king","queen")
print("\nSimilarity between king and queen:", round(sim,4))
words = ["apple","banana","grape","car"]
print("\nOdd one out:", model.doesnt_match(words))
```

2. Use dimensionality reduction (e.g., PCA or t-SNE) to visualize word embeddings for Q 1. Select 10 words from a specific domain (e.g., sports, technology) and visualize their embeddings. Analyze clusters and relationships. Generate contextually rich outputs using embeddings. Write a program to generate 5 semantically similar words for a given input.

```
import gensim.downloader as api
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

m = api.load("word2vec-google-news-300")
w=["computer","algorithm","software","hardware","ai","cloud","database","network","cybersecurity","encryption"]
p = PCA(n_components=2).fit_transform([m[i] for i in w])
for a,(x,y) in zip(w,p):
    plt.scatter(x,y)
    plt.text(x,y,a)

plt.title("Word Embeddings")
plt.grid()
plt.show()
for i,j in m.most_similar("ai",topn=5):
    print(i,round(j,4))
```

3. Train a custom Word2Vec model on a small dataset. Train embeddings on a domain-specific corpus (e.g., legal, medical) and analyze how embeddings capture domain-specific semantics.

```
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
```

```

corpus = ["The patient diagnosed with diabetes", "Insulin treats diabetes", "He has heart diseases", "Unhygiene
leads to infections",
          "Vaccines helps cure disease", "The person suffered from cancer"]
data = [s.lower().split() for s in corpus]
model = Word2Vec(data, vector_size=50, window=5, min_count=1)
print("Similar to diabetes:")
for w,s in model.wv.most_similar("diabetes",topn=5):
    print(w,round(s,3))

words = list(model.wv.index_to_key)
vectors = model.wv[words]
tsne = TSNE(n_components=2, perplexity=5, random_state=0).fit_transform(vectors)
for i,w in enumerate(words):
    x,y = tsne[i]
    plt.scatter(x,y)
    plt.text(x,y,w)
plt.title("Medical Word Embeddings")
plt.grid()
plt.show()

```

-
- 4. Use word embeddings to improve prompts for Generative AI model. Retrieve similar words using word embeddings. Use the similar words to enrich a GenAI prompt. Use the AI model to generate responses for the original and enriched prompts. Compare the outputs in terms of detail and relevance.**

```

import gensim.downloader as api
from transformers import pipeline

model = api.load("glove-twitter-25")
original = "The forest is beautiful"
synonyms = [w for w,s in model.most_similar("beautiful",topn=3)]
enriched = original + " " + " ".join(synonyms)
gen = pipeline("text-generation", model="gpt2")

out1 = gen(original, max_new_tokens=30)[0]['generated_text']
out2 = gen(enriched, max_new_tokens=30)[0]['generated_text']
print("\nOriginal Output:")
print(out1)
print("\nEnriched Output:")
print(out2)

```

-
- 5. Use word embeddings to create meaningful sentences for creative tasks. Retrieve similar words for a seed word. Create a sentence or story using these words as a starting point. Write a program that: Takes a seed word. Generates similar words. Constructs a short paragraph using these words.**

```

import gensim.downloader as api
import random

model = api.load("glove-wiki-gigaword-50")

```

```
seed = input("Enter seed word: ").lower()
words = [w for w,s in model.most_similar(seed,topn=5)]
```

```
story = f""" The story of the {seed} began in a distant land. It met a {random.choice(words)} near a mysterious
{random.choice(words)}.
The {seed} discovered {random.choice(words)} and continued the adventure with {random.choice(words)}
leading to {random.choice(words)}. """
print("\nGenerated Story:\n")
print(story)
```

6. Use a pre-trained Hugging Face model to analyze sentiment in text. Assume a real-world application, Load the sentiment analysis pipeline. Analyze the sentiment by giving sentences to input.

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
while True:
    text = input("Enter sentence (or exit): ")
    if text.lower() == "exit":
        break
    res = classifier(text)[0]
    print("Sentiment:", res['label'], "Score:", round(res['score'],3))
```

7. Summarize long texts using a pre-trained summarization model using Hugging face model. Load the summarization pipeline. Take a passage as input and obtain the summarized text.

```
from transformers import pipeline

summarizer = pipeline("summarization", model="t5-small")
text = input("Enter the text you want to summarize:\n")
summary = summarizer(text, max_length=100, min_length=30, do_sample=False)
print("\nSummarized Text:")
print(summary[0]['summary_text'])
```

8. Install langchain, cohere (for key), langchain-community. Get the api key(By logging into Cohere and obtaining the cohere key). Load a text document from your google drive . Create a prompt template to display the output in a particular manner.

```
#!pip install langchain-community
from langchain_core.prompts import PromptTemplate
from langchain_community.llms import HuggingFacePipeline
from transformers import pipeline

pipe = pipeline("text-generation", model="gpt2", max_new_tokens=40)
llm = HuggingFacePipeline(pipeline=pipe)
with open("Ai.txt","r") as f:
    text = f.read()
```

```

text = text[:800]
prompt = PromptTemplate(
    input_variables=["t"],
    template="Summarize this text:\n {t}"
)
chain = prompt | llm
print(chain.invoke({"t": text}))

```

-
- 9. Take the Institution name as input. Use Pydantic to define the schema for the desired output and create a custom output parser. Invoke the Chain and Fetch Results. Extract the below Institution related details from Wikipedia: The founder of the Institution. When it was founded. The current branches in the institution . How many employees are working in it. A brief 4-line summary of the institution.**

```

#!/pip install wikipedia
from pydantic import BaseModel
import wikipedia

class Institution(BaseModel):
    name:str
    founder:str
    founded:str
    branches:str
    employees:str
    summary:str
name = input("Enter Institution Name: ")
page = wikipedia.page(name)
summary = ". ".join(page.summary.split(".")[4]) + "."

data = Institution(
    name = page.title,
    founder = "Check Wikipedia",
    founded = "Check Wikipedia",
    branches = "Multiple",
    employees = "N/A",
    summary = summary
)
print(data.model_dump_json(indent=4))

```

-
- 10. Build a chatbot for the Indian Penal Code. We'll start by downloading the official Indian Penal Code document, and then we'll create a chatbot that can interact with it. Users will be able to ask questions about the Indian Penal Code and have a conversation with it.**

```

#!/pip install PyPDF2
import PyPDF2
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

```

```

def get_pdf_text(file):
    text=""
    with open(file,"rb") as f:
        pdf=PyPDF2.PdfReader(f)
        for p in pdf.pages[:30]:
            text+=p.extract_text()
    return text
print("Loading IPC...")
text=get_pdf_text("ipc.pdf")
sections=[s.strip() for s in text.split("Section") if len(s)>20]

while True:
    q=input("\nAsk IPC (or exit): ")
    if q.lower()=="exit":
        break
    vec=TfidfVectorizer()
    mat=vec.fit_transform(sections+[q])
    sim=cosine_similarity(mat[-1],mat[:-1]).flatten()
    print("\nResult:",sections[sim.argmax()][:400],"...")

```